Personal AI Running Coach – Projektüberblick

Gruppe AI: Felix Walter, Samuel Ringl, Lukas Herk · LV Human-AI Interaction I Stand: 10.11.2026

1. Projektregeln (LV)

- KI mit Perception, Reasoning, Action + Methode zur Anpassung an neue Information
- Interface für menschlichen Input (Perception) und Feedback (Action)
- Prototyp: alle HAI-relevanten Komponenten, kein vollständiges System

2. KI-Konzept (aus der Präsentation)

- **Was:** Lokales LLM als Laufcoach. Plant und erklärt das Training, regelbasierte Guardrails prüfen jeden Plan.
- **Perception:** Onboarding-Fragebogen, Session-Logs (Distanz, Zeit, Anstrengung/RPE), Chat-Nachrichten, GPX/FIT-Dateien.
- **Reasoning:** Nach jeder Session wird das Profil aktualisiert. Das LLM schlägt den Plan für die nächste Woche vor, die Guardrails lehnen unsichere Pläne ab.
- **Action:** Aktualisierter Wochenplan, Tipp für die nächste Session, Begründung jeder Änderung, Warnungen (z. B. bei wiederholtem Schmerz).
- **Modelle:** Vortrainierte Open-Weight-Modelle über Ollama (Llama 3.1 8B, Qwen 2.5 7B, Mistral 7B), kein eigenes Training.
- **Tests:** Simulierte User: schneller Fortschritt, ausgelassene Läufe, Verletzung, lange Pause.
- **Daten:** Profil, Läufe und Chat lokal. Die jüngste Historie fließt in den Prompt.
- **Privacy:** Alles bleibt privat und lokal. Nur Fragen, die das lokale Modell nicht beantworten kann, gehen an einen externen Dienst, ohne personenbezogene Daten und nach Bestätigung durch den User.
- **Was die KI vom Menschen braucht:** Ziel, Level, Einschränkungen (Zeit, Gelände, Verletzungen), Feedback nach jeder Session.
- **Was sie erklären muss:** Warum sich der Plan geändert hat, was extern gesendet würde, den aktuellen Trainingsstatus, wie sicher sie ist und welche Daten lokal bleiben.
- **Allgemeinwissen:** Trainingsprinzipien (schrittweise Laststeigerung, Erholung, Ruhetage, typische Progression), umgesetzt in Prompt und Guardrails. Keine Daten anderer User.
- **Erwartete Situationen:** stetiger Fortschritt, verpasste oder harte Sessions, wechselnde Zeitpläne.
- **Unerwartete Situationen:** Schmerzmeldungen, unrealistische Ziele, unmögliche Logs, lange Pausen, Fragen ohne Bezug zum Training.

3. Roadmap

- **Erste Schritte:** Ollama aufsetzen und Modelle vergleichen. Profil, Planformat, Guardrails und DB definieren. Onboarding bauen.
- **M1 – 15.12.2026:** Onboarding, erster Plan, Logging (Formular, GPX/FIT), lokale Speicherung.
- **M2 – 11.01.2027:** Adaptions-Loop, Erklärungen im Chat, externer Fallback, Tests mit simulierten Usern.
- **Ressourcen:** Laptop mit ca. 16 GB RAM, freie Tools, API-Key (nur für den Fallback), keine Laborarbeit.

4. Tech-Stack (beschlossen, ersetzt die Angaben in den Folien)

Anforderung: muss auf Notebook und Handy funktionieren.

- **Frontend:** Responsive PWA. Empfohlen: React + Vite + TypeScript, Tailwind, shadcn/ui, TanStack Query, react-hook-form + zod, Recharts, vite-plugin-pwa. Die TS-Typen werden aus dem OpenAPI-Schema von FastAPI generiert. Der Chat streamt per SSE.
- **Backend:** Python + FastAPI, Pydantic v2, SQLModel. Import mit gpxpy und fitdecode. Tests mit pytest, Tooling mit uv und ruff.
- **LLM:** Ollama am Laptop, angesprochen über ein provider-neutrales `LLMClient`-Interface. Die Ausgabe ist JSON nach Schema.
- **Cloud-Fallback:** optionales Add-on hinter einem Feature-Flag. Ablauf: PII-Filter, dann Dialog mit dem exakten Payload, dann Bestätigung durch den User, dann der API-Call. Jeder Call wird im Audit-Log festgehalten.
- **Netzwerk:** Der Laptop dient als Server. Das Handy greift über Tailscale Serve zu, das liefert das HTTPS, das Service Worker und PWA voraussetzen. Dazu ein einfacher Token-Login.
- **Datenbank:** SQLite.
  - Warum: kein Server, eine einzige Datei, passt zu Privacy by Design, kleine Datenmenge, nur ein Schreiber.
  - Alternativen: PostgreSQL wäre der Schritt nach dem Prototyp. DuckDB eignet sich höchstens für Analysen. sqlite-vec nur, falls ihr RAG baut.
  - Durch SQLModel ist ein späterer Wechsel einfach.

Backend-Module:

- Ingest (GPX/FIT)
- Profil-Updater: deterministisch, berechnet Wochenumfang, Session-Load (Dauer × RPE), ACWR, RPE-Trend und Schmerz-Zähler
- Planner + Guardrails
- Chat-Router: Intents `training_question` / `plan_change` / `off_topic` / `needs_external`

Guardrails (Beispiele):

- Wochenumfang steigt maximal um ca. 10 %
- Mindestens 1 Ruhetag
- Keine zwei harten Einheiten hintereinander
- Long Run höchstens ca. 30 % des Wochenumfangs
- Bei Schmerz: Warnung, Last reduzieren, Arztbesuch empfehlen
- Bei einer Verletzung bekommt das LLM die Verletzung zurück und plant neu (max. 2 Versuche). Danach greift ein deterministischer Fallback-Plan.

Konfidenz: wird aus der Datenlage abgeleitet (Anzahl der Logs, Lücken, RPE-Varianz), nicht vom LLM geschätzt.

DB-Tabellen: user_profile, sessions, tracks, weekly_plans (versioniert, mit Begründung), plan_changes, chat_messages, external_requests (Audit).

Tests: Jede Persona wird über 8 simulierte Wochen mit allen drei Modellen durchgespielt. Gemessen werden JSON-Validität, Guardrail-Verletzungen, Latenz und Erklärungsqualität

(manuell).

Repo-Struktur: `running-coach/{backend, frontend}` (Monorepo).

Meilensteine technisch:

- **M1:** Datenmodell, Onboarding, Import, erster Plan mit Guardrails, PWA am Handy über Tailscale erreichbar.
- **M2:** Profil-Updater/Adaptions-Loop, Chat mit Streaming und Erklärungen, Charts, Persona-Tests, optional Cloud-Fallback.

Das Architekturdiagramm wurde in draw.io erstellt.

5. Zusammenarbeit

- **GitHub:** Mitglieder über Settings → Collaborators einladen. Fremde können bei einem Public Repo nicht pushen oder mergen, aber Code kopieren. Empfehlung: Repo privat halten, mindestens bis zur Benotung.
- **`.gitignore`:** `.env`, `*.db`, `node_modules/`. Keine API-Keys committen.
- **Branch-Schutz für `main`:** PR + Review verlangen. Bei privaten Repos braucht das GitHub Pro, das es über das Student Pack gratis gibt.
- **Repo nicht in iCloud ablegen** (Sync-Probleme mit `.git`). Besser lokal, z. B. `~/dev/running-coach`.
- **Gemeinsame Dokumente in Google Drive.** Google Docs in iCloud lassen sich nicht gemeinsam bearbeiten.

6. Offene Entscheidungen

1. **Planungsansatz:** LLM plant und Guardrails prüfen (so in den Folien), oder ein Algorithmus plant und das LLM erklärt (Empfehlung: sicherer und besser erklärbar). Vor M1 entscheiden.
2. **Frontend-Framework:** React (empfohlen) oder Svelte.
3. **Externer API-Anbieter** für den Fallback.
4. **Plan-Rhythmus:** fester Wochenplan oder rollierender Plan.
5. **Herzfrequenz:** nutzen, falls GPX/FIT sie enthalten?
6. **Offline-Logging** am Handy, wenn der Laptop aus ist (Zwischenspeicher in der PWA).
7. **Folien aktualisieren:** Dort stehen noch Streamlit und „nur Laptop“.